

# Everything About Itnavideo

Complete Technical Blueprint: Remotion Engine, Groq Whisper LPU, AWS Serverless Infrastructure, Video Pipelines, Pricing, and Issue Diagnostics.

TARGET PROJECT

Itnavideo (SaaS Video Creation Engine)

CORE STACK

Next.js 15, Remotion, Groq AI, AWS Lambda & S3

PRIMARY FOCUS

Remotion, Groq Whisper, AWS Rendering, Billing

TARGET AUDIENCE

Founder & Lead Engineers (Deep Comprehension)

# Table of Contents

---

- **Section 1: What is Itnavideo? (High-Level Overview)** — Core Mission, What it does, How the stack works together.
- **Section 2: Deep Dive into Remotion** — What is Remotion? When, where, and why we use it? How it renders video using React.
- **Section 3: Deep Dive into Groq & Whisper AI** — LPU Architecture, Speech-to-Text speed, word-level timestamps, Gemini fallback.
- **Section 4: Deep Dive into AWS Infrastructure** — S3 Storage, Lambda distributed rendering, IAM credentials, concurrency.
- **Section 5: Video Types & Template Ecosystem** — 9:16 Reels vs 16:9 Landscape YouTube Subtitles, the 5 Real Creator Styles.
- **Section 6: Credit & Billing Architecture** — Mathematical rules, deductions, 1 credit per 2 min logic, refund on failure.
- **Section 7: Step-by-Step Lifecycle of a Video Request** — Exact journey of 1 video render from upload to final MP4 download.
- **Section 8: Timing, Performance & Bottlenecks** — How many seconds each step takes, where delays happen, how to optimize.
- **Section 9: Complete Troubleshooting & Issue Resolution Guide** — Groq 429, Lambda OOM, S3 NoSuchKey, font glitches, and fixes.

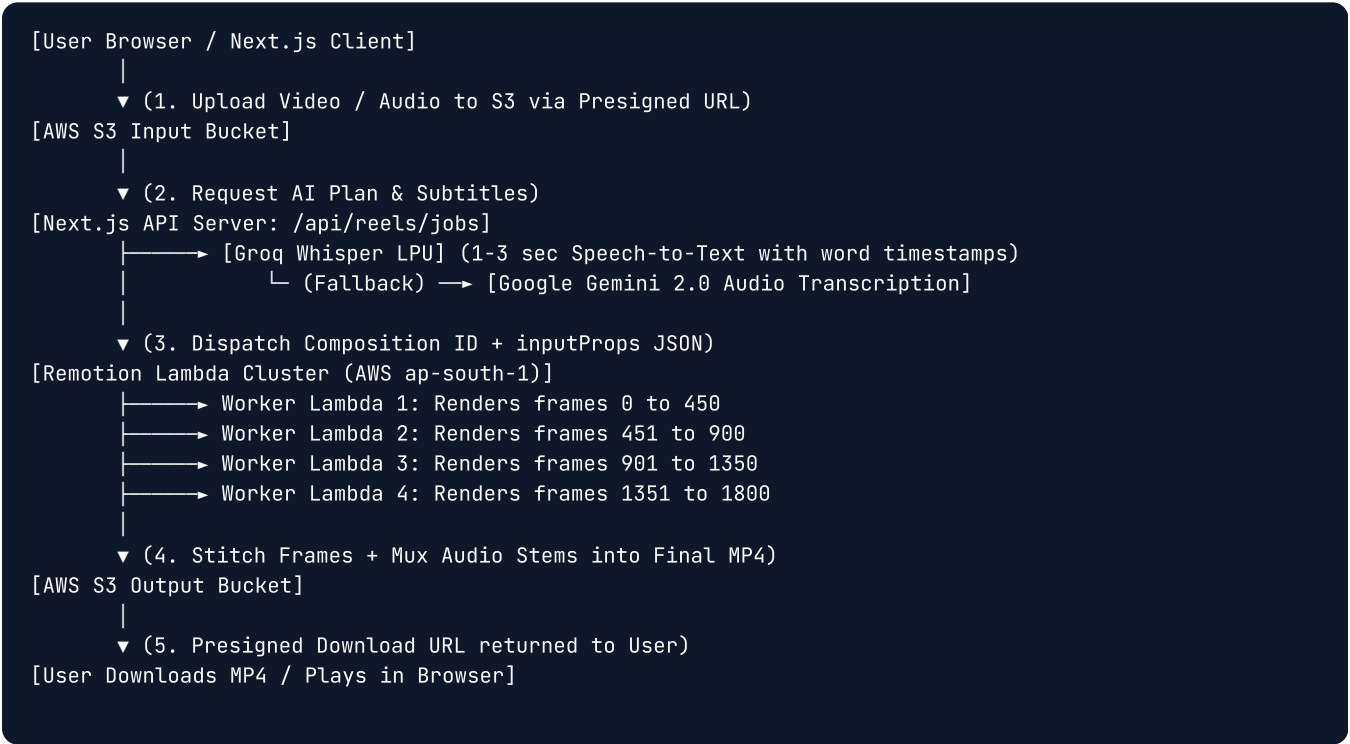
# 1. What is Itnavideo? (High-Level Overview)

Itnavideo is an automated, AI-powered video rendering SaaS platform designed for content creators, podcasters, educators, and social media marketers. It eliminates the traditional video editing bottlenecks:

Core Capabilities

- Auto Word-Synced Kinetic Captions:** Creates animated, high-retention subtitles synced to the exact millisecond of human speech.
- YouTube Subtitle Generator (16:9 Landscape):** Adds broadcast-quality subtitles to long-form YouTube videos up to 15 minutes.
- Studio Audio Enhancer:** Removes room echo, hiss, and hum while preserving voice clarity.
- Fast Cloud Video Rendering:** Exports crisp 1080p Full HD MP4 videos in seconds without needing an expensive GPU laptop.

## High-Level Architecture Diagram



## 2. Deep Dive into Remotion

### 2.1 What is Remotion?

**Remotion** is a framework that allows developers to create MP4 videos programmatically using **React, HTML, CSS, SVG, and Canvas**. Instead of learning complex FFmpeg command-line scripts or writing Python MoviePy scripts, you write standard React components with props, state, and CSS animations.

### 2.2 When and Why do we use Remotion in Itnvideo?

#### Why NOT pure FFmpeg or traditional video editors?

- **Dynamic Typography:** Creating word-by-word kinetic highlight effects (like MrBeast or Ali Abdaal styles) in pure FFmpeg requires thousands of lines of complicated ASS subtitle filter graphs that break easily.
- **Component Reusability:** In Remotion, a subtitle pill, a progress bar, or a watermark is simply a React component ( `<MotionCaptionRenderer />` ).
- **Live In-Browser Preview:** Remotion provides `@remotion/player` which runs client-side in the browser. Users can preview animations in real-time at 60 FPS before spending a single credit on rendering!

### 2.3 Where is Remotion in the Codebase?

| Folder / File Path                                           | Role & Functionality                                                                                                                                                        |
|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>remotion/index.tsx</code>                              | <b>Root Entry Point:</b> Calls <code>registerRoot(RemotionRoot)</code> . Registers all video compositions with Remotion.                                                    |
| <code>remotion/templates/AUTO_CAPTION_GENERATOR/</code>      | <b>Landscape Subtitle Engine:</b> The composition used for the YouTube Subtitle Generator (1920×1080) and 9:16 Shorts.                                                      |
| <code>remotion/components/MotionCaptionRendererer.tsx</code> | <b>Word-Level Kinetic Renderer:</b> Calculates current frame time and renders active words with bounce, pop, glow, and color highlights.                                    |
| <code>remotion/types/subtitles.ts</code>                     | <b>Style Definitions:</b> Contains preset configs for <i>Minimal Clean</i> , <i>Cinematic Docu</i> , <i>Studio Podcast</i> , <i>Bold Creator</i> , <i>Netflix Classic</i> . |
| <code>app/api/reels/jobs/route.ts</code>                     | <b>Lambda Dispatcher:</b> Calls <code>renderMediaOnLambda()</code> from <code>@remotion/lambda/client</code> to trigger rendering on AWS.                                   |
| <code>app/api/reels/jobs/status/route.ts</code>              | <b>Polling Endpoint:</b> Calls <code>getRenderProgress()</code> to return progress percentage (0% to 100%) to the frontend progress bar.                                    |

### 2.4 How Remotion Works Internally

1

Frame Concept

Remotion treats video as a series of static frames. A 30 FPS video has 30 frames per second. A 60-second video has exactly 1,800 frames ( `0` to `1799` ).

## 2 **useCurrentFrame() Hook**

Every React component inside a composition calls `const frame = useCurrentFrame();`. If `frame === 30`, Remotion calculates what the UI looks like at exactly 1.0 second.

## 3 **Headless Chromium Capture**

Remotion opens a headless Chromium browser instance in AWS Lambda. It advances the frame counter one by one, takes a screenshot of the DOM/canvas, and passes the raw buffer to an internal FFmpeg encoder.

### 3. Deep Dive into Groq & Whisper AI

#### 3.1 What is Groq and what is an LPU?

**Groq** is an ultra-fast AI inference platform. Unlike traditional GPUs (NVIDIA A100/H100) that use high-bandwidth memory architectures, Groq invented the **LPU (Language Processing Unit)**, a custom chip engineered specifically for tensor processing with deterministic latency.

#### 3.2 Why Groq Whisper instead of OpenAI Whisper API?

| Comparison Metric               | Standard OpenAI Whisper API           | Groq Whisper (whisper-large-v3)                  |
|---------------------------------|---------------------------------------|--------------------------------------------------|
| Inference Latency (1 min audio) | 15 to 30 seconds                      | <b>1.2 to 2.8 seconds</b> (10x - 20x faster)     |
| Word-Level Timestamps           | Supported (slower)                    | <b>Instant word-level millisecond accuracy</b>   |
| Cost per Hour of Audio          | ~\$0.36 / hr                          | <b>~\$0.02 to \$0.05 / hr</b> (Over 80% savings) |
| Impact on User Experience       | User waits ~30s just to see subtitles | <b>Captions appear almost instantly</b>          |

#### 3.3 Where is Groq in the Codebase?

- Defined in `app/api/reels/jobs/route.ts` inside `transcribeMediaUrlWithGroq()`.
- Audio is extracted from the video stream and sent with model `whisper-large-v3` using `response_format: "verbose_json"` and `timestamp_granularities: ["word"]`.
- Dual Engine Resiliency:** If Groq returns an error or HTTP 429 rate limit, `ltnavideo` automatically initiates `transcribeMediaWithGeminiFallback()` using Google Gemini 2.0 Audio multimodal processing. The user never experiences an error screen!

## 4. Deep Dive into AWS Infrastructure

### 4.1 Why AWS Lambda for Remotion?

Rendering video locally on a standard server or your laptop takes 5 to 15 minutes and maxes out the CPU/RAM. **Remotion Lambda** solves this by dividing the video into chunks and rendering them across dozens of serverless Lambda instances concurrently.

**Distributed Rendering Math:**

A 15-minute video at 30 FPS has **27,000 frames**.

- If rendered on 1 CPU: ~30 to 45 minutes.
- On Remotion Lambda with concurrency = 10: 10 Lambdas render 2,700 frames each simultaneously.
- **Result:** Video renders in ~3 to 4 minutes!

### 4.2 AWS Components in Itnavideo

| AWS Service     | Environment Variable          | Purpose                                                                             |
|-----------------|-------------------------------|-------------------------------------------------------------------------------------|
| AWS Region      | REMOTION_AWS_REGION           | Configured to <code>ap-south-1</code> (Mumbai) for ultra-low latency in India/Asia. |
| S3 Bucket       | REMOTION_LAMBDA_BUCKET_NAME   | Stores deployed Remotion bundle, user uploads, audio stems, and output MP4 files.   |
| Lambda Function | REMOTION_LAMBDA_FUNCTION_NAME | Custom Remotion container image with Chromium + FFmpeg pre-installed.               |
| Serve URL       | REMOTION_LAMBDA_SERVE_URL     | Pre-bundled webpack site index hosted on S3 containing all React video code.        |
| Concurrency     | REMOTION_LAMBDA_CONCURRENCY   | Defines how many parallel Lambda workers run simultaneously per render job.         |

## 5. Video Types & Template Ecosystem

Itnavideo supports distinct video modes optimized for different creator workflows:

| Video Mode                 | Aspect Ratio & Specs               | Max Duration      | Pricing Model          |
|----------------------------|------------------------------------|-------------------|------------------------|
| YouTube Subtitle Generator | 16:9 Landscape (1920×1080), 30 FPS | 15 Minutes (900s) | 1 Credit per 2 Minutes |
| Auto Caption Generator     | 9:16 Vertical (1080×1920), 30 FPS  | 90 Seconds        | 1 Credit flat          |
| Compare Explainer          | 9:16 Vertical Split-Screen, SFX    | 60 Seconds        | 1 Credit flat          |
| Faceless Long Video        | 16:9 Landscape AI Narrative        | 5 Minutes         | Tiered Credits         |

### The 5 Real Creator Subtitle Styles

- Minimal Clean** (Ali Abdaal / Veritasium): Inter Sans font, translucent dark pill background, subtle highlight, zero visual clutter.
  - Cinematic Docu** (Vox / Johnny Harris): Montserrat Bold font, warm cinema off-white ( #F5F5F0 ) with golden yellow keyphrase emphasis ( #FDE047 ).
  - Studio Podcast** (Diary of a CEO / Huberman Lab / Joe Rogan): Poppins Bold font, matte black rounded box, maximum readability on complex background video.
  - Bold Creator** (MrBeast / MKBHD): Montserrat ExtraBold font, black stroke outline, high-energy pop animations with vibrant yellow highlights.
  - Netflix Classic** (Documentary / Cinema): Roboto Condensed font, broadcast yellow standard ( #FFDE00 ) with drop shadow.

## 6. Credit & Billing Architecture

Credit deduction is strictly managed in `lib/billing/creditPricing.ts` to prevent abuse and ensure transactional safety:

### 6.1 The Mathematical Pricing Formula

```
export function calculateYouTubeSubtitleCreditUnits(durationSeconds: number): number {
  const safeSeconds = Math.max(1, Math.min(15 * 60, Math.round(durationSeconds)));
  // 1 credit for every 2 minutes (120 seconds block)
  const creditUnits = Math.max(1, Math.ceil(safeSeconds / 120)) * CREDIT_UNITS_PER_CREDIT;
  return creditUnits;
}
```

### 6.2 Credit Breakdown for YouTube Subtitle Generator

| Video Duration                    | 2-Minute Blocks | Credits Charged              |
|-----------------------------------|-----------------|------------------------------|
| 0 to 120 seconds (0 - 2 mins)     | 1 block         | 1 Credit                     |
| 121 to 240 seconds (2 - 4 mins)   | 2 blocks        | 2 Credits                    |
| 241 to 360 seconds (4 - 6 mins)   | 3 blocks        | 3 Credits                    |
| 541 to 600 seconds (9 - 10 mins)  | 5 blocks        | 5 Credits                    |
| 841 to 900 seconds (14 - 15 mins) | 8 blocks        | 8 Credits (Max duration cap) |

**Automatic Refund Guarantee**

If any render fails due to a network glitch, transcription failure, or Lambda timeout, the backend catches the error inside `app/api/reels/jobs/route.ts` and immediately releases the reserved credits back to the user's balance. No credits are ever lost on failed jobs.

## 7. Step-by-Step Lifecycle of a Video Request

---

### 1 User Uploads Video File

Frontend requests a presigned S3 upload URL from `/api/media/presign`. The video streams directly from the user's browser to the AWS S3 bucket (bypassing the Next.js server to avoid payload size limits).

### 2 Job Creation & Credit Reservation

Frontend triggers `POST /api/reels/jobs` with the S3 URL, duration, and chosen subtitle style. The backend verifies the user's balance and reserves the required credits.

### 3 Groq Whisper Transcription

`transcribeMediaUrlWithGroq()` runs. Audio is extracted and transcribed into word-level timestamps in ~1.5 to 3 seconds.

### 4 Style & Subtitle Formatting

The words are grouped into subtitle clauses based on punctuation and reading pace, then mapped to the selected creator style preset.

### 5 Remotion Lambda Dispatch

The backend calls `renderMediaOnLambda()`. Lambda instances spin up and render frame ranges concurrently.

### 6 Frontend Polling & Progress Updates

Client polls `GET /api/reels/jobs/status?renderId=...` every 2 seconds. Remotion updates progress percentage from 1% to 100%.

### 7 Stitching & S3 MP4 Export

The main Lambda worker stitches all rendered frame chunks, merges the original audio track, and writes the final MP4 to S3.

### 8 Presigned Download & Credit Finalization

Backend confirms render success, finalizes credit deduction, and returns a secure presigned download link to the user.

## 8. Timing, Performance & Bottlenecks

---

Here is an empirical breakdown of how long each stage takes for a standard 60-second video vs a 10-minute long video:

| Pipeline Stage                | 60s Reel Duration       | 10m Long Video Duration   | Where the Time is Spent                               |
|-------------------------------|-------------------------|---------------------------|-------------------------------------------------------|
| <b>S3 Direct Upload</b>       | 2 – 6 seconds           | 15 – 45 seconds           | Depends entirely on user's internet upload bandwidth. |
| <b>Groq Whisper AI</b>        | <b>1.5 – 3 seconds</b>  | <b>6 – 12 seconds</b>     | Groq LPU chip transcription & word timestamps.        |
| <b>Lambda Cold Boot</b>       | 8 – 15 seconds          | 8 – 15 seconds            | AWS container spin-up & Remotion bundle fetch.        |
| <b>Frame Rendering</b>        | 12 – 25 seconds         | 90 – 160 seconds          | Headless Chromium capturing React DOM elements.       |
| <b>Stitching &amp; Upload</b> | 3 – 6 seconds           | 12 – 20 seconds           | FFmpeg concatenating video chunks and encoding H.264. |
| <b>TOTAL END-TO-END</b>       | <b>~28 – 55 seconds</b> | <b>~2.5 – 4.5 minutes</b> | 100% automated in the cloud without user CPU load.    |

## 9. Complete Troubleshooting & Issue Resolution Guide

### Issue 1: "NoSuchKey" or S3 404 in first 20 seconds of render

**Cause:** When Remotion Lambda starts, the main worker has not written `progress.json` to S3 yet during container spin-up.

**Resolution:** In `app/api/reels/jobs/status/route.ts`, attempts 0 to 10 treat S3 `NoSuchKey` as `status: 'rendering'` with `progress = 5%` instead of throwing a failure error.

### Issue 2: Groq HTTP 429 (Rate Limit Exceeded)

**Cause:** High traffic or burst requests exceeding Groq's tokens-per-minute threshold.

**Resolution:** The backend in `app/api/reels/jobs/route.ts` catches Groq 429 errors and automatically switches to `transcribeMediaWithGeminiFallback()`. No user intervention needed.

### Issue 3: Remotion Lambda Timeout or Out of Memory (OOM)

**Cause:** Long videos (e.g. 15 minutes) render thousands of frames. If concurrency is too low (e.g. 1) or memory is 2048MB, Lambda times out at 900 seconds.

**Resolution:** 1. Ensure Lambda memory is set to at least **3008 MB**.

2. Set `REMOTION_LAMBDA_CONCURRENCY=4` or higher for long-form videos.

3. Do NOT pass both `framesPerLambda` and `concurrency` simultaneously (Remotion Lambda SDK throws an error if both are provided).

### Issue 4: Fonts not rendering correctly in exported MP4

**Cause:** Headless Chromium in AWS Lambda does not have local Windows or Mac fonts installed.

**Resolution:** All fonts in Itnavideo must be loaded via `@remotion/google-fonts` (e.g. `loadFont('Inter')`, `loadFont('Montserrat')`) inside the composition template. This guarantees cross-platform font consistency.

### Issue 5: S3 CORS Error during direct browser upload

**Cause:** The S3 bucket CORS configuration is missing `PUT`, `POST`, or `x-amz-meta-*` headers.

**Resolution:** Verify S3 CORS rule allows `AllowedOrigins: ["*"]`, `AllowedMethods: ["GET", "PUT", "POST", "HEAD"]`, and `AllowedHeaders: ["*"]`.